

Optimization for Learning

Exercises

Anders Hansson and Martin Andersen

May 6, 2019

Maximum entropy for page rank

Exercise 1 In this exercise we consider the static ranking of web pages. To formalize this we let $V = \mathbf{N}_n$ be a set of web pages or vertices in a graph $G = (V, E)$, where the edge set $E \subset V \times V$ contains directed edges (i, j) describing that there is a link from web page i to web page j . The most well-known way to model the ranking is called PageRank. It uses a Markov Chain model for which the probabilities of the out-links from a page are equal. Let p_{ij} be the transition probability that a user at page i goes to page j . These probabilities are then given by

$$p_{ij} = \begin{cases} \frac{1}{d_i}, & (i, j) \in E \\ 0, & (i, j) \notin E \end{cases}$$

where d_i is the out-degree of node i , i.e., the number of edges $(i, j) \in E$ where $j \in V$. The PageRank is then defined as the stationary distribution of the Markov Chain, i.e. as the solution of $x^T = x^T P$, where $P \in \mathbf{R}^{n \times n}$ is the matrix of transition probabilities p_{ij} at position i, j and zeros otherwise.

Another way to model the transition probabilities is based on a network flow approach. Let y_{ij} be the number of users following link $(i, j) \in E$ per unit time. Assume that the web traffic is in a state of equilibrium so that the traffic out of a node is equal to the traffic in per unit time, i.e.

$$\sum_{j:(i,j) \in E} y_{ij} = \sum_{j:(j,i) \in E} y_{ji}, \quad i \in V$$

Define

$$H_j = \sum_{i:(i,j) \in E} y_{ij}, \quad j \in V$$

which is the number of "hits" per unit time at node j . Also define

$$Y = \sum_{(i,j) \in E} y_{i,j} = \sum_{j \in V} H_j$$

which is the total number of hits per unit time. We now define the probabilities $p_{ij} = y_{i,j}/Y$, and we may then write the equilibrium condition as

$$\sum_{j:(i,j) \in E} p_{ij} = \sum_{j:(j,i) \in E} p_{ji}, \quad i \in V$$

Notice that these probabilities are not the transition probabilities discussed before. They are obtained by normalization with $\sum_{j:(i,j) \in E} p_{ij}$.

One solution to the above equations is

$$p_{ij} = \frac{H_i}{Y d_i}, \quad (i, j) \in E$$

which agrees with the traditional PageRank model as described above after normalization. However, there are many more solutions to the equilibrium condition, which can be interpreted as moment constraints. We will now investigate the maximum entropy solution under the moment constraints.

1. Formulate the maximum entropy problem and show that $p_{ij} = \exp(-\lambda_i + \lambda_j)/\Phi(\lambda)$, $(i, j) \in E$, where $\Phi(\lambda) = \sum_{(i,j) \in E} \exp(-\lambda_i + \lambda_j)$, and where $\lambda \in \mathbf{R}^n$ are the natural parameters. Notice that you get different answers depending on how you rewrite the equilibrium conditions as equations with a zero right hand side.
2. Let $a_i = \exp(-\lambda_i)$, $i \in V$ and let $A = \mathbf{diag}(a_i)$. Moreover, define the sparse matrix $M \in \mathbf{R}^{n \times n}$ as

$$M_{ij} = \begin{cases} \Phi(\lambda)^{-1}, & (i, j) \in E \\ 0, & (i, j) \notin E \end{cases}$$

Show that $P = AMA^{-1}$.

3. The so-called iterative scaling or matrix balancing approach for finding the optimal solution to the maximum entropy problem is based on the expression above for P and the fact that $\Phi(\lambda) = \sum_{(i,j) \in E} a_i/a_j$. It is an iterative algorithm and can be described as follows, where superscripts are used to denote iteration index.

Given a tolerance $\epsilon > 0$, initialize as $a_i^{(0)} = 1.0$, $Z^{(0)} = \sum_{(i,j) \in E} 1$, and then repeat starting with $k = 0$:

1. $p_{ij}^{(k)} = a_i^{(k)} / (a_j^{(k)} Z^{(k)})$, $(i, j) \in E$
2. $\rho_i^{(k)} = \sum_{j:(i,j) \in E} p_{ij}^{(k)}$, $i \in V$
3. $\sigma_i^{(k)} = \sum_{j:(j,i) \in E} p_{ji}^{(k)}$, $i \in V$
4. $\eta_i^{(k)} = (\sigma_i^{(k)} / \rho_i^{(k)})^{1/2}$, $i \in V$
5. If $1 - \epsilon \leq \eta_i^{(k)} \leq 1 + \epsilon$ for all $i \in V$ go to Step 7.
6. For those $i \in V$ for which $1 - \epsilon \leq \eta_i^{(k)} \leq 1 + \epsilon$ does not hold, let $a_i^{(k+1)} = a_i^{(k)} \eta_i^{(k)}$, increase k , and then go to Step 1.
7. Break if $1 - \epsilon \leq \sum_{(i,j) \in E} p_{ij} \leq 1 + \epsilon$. Otherwise let $Z^{(k+1)} = \sum_{(i,j) \in E} a_i^{(k)} / a_j^{(k)}$, increase k , and then go to Step 1

Implement the above algorithm and test it on random graphs generated by the Matlab function `webgraph.m`. This function generates the edges for a graph with `dim` edges, for which the first node is connected to all other nodes in both directions. In this way the constructed graph is strongly connected, modeling the fact that every page potentially can be visited from every page despite that there is not a link from the current page. The other nodes are connected to at most 14 other nodes. Notice that you are not asked to compute the page rank. It is enough to compute the un-normalized p_{ij} for the edges. Make a plot of

the computational time as a function of the variable `dim`. Average over 10 random graphs for each dimension. You should at least consider graphs for which `dim=10000`. You should use $\epsilon = 10^{-12}$.

4. It is also possible to state the above maximum entropy problem as a conic optimization problem as described in the lecture notes. Do this and use the conic solver Mosek to solve exactly the same problems as above and compare the solution times. How big problems can you solve with the two different solvers? Make sure to compare exactly the same problems, i.e. do not use different random realizations for the two different solvers but the same realizations of the edges for the graph. To make the comparison fair try to choose tolerances in Mosek so that $1 - \epsilon \leq \sum_{(i,j) \in E} p_{ij} \leq 1 + \epsilon$, or do it the other way around, i.e. choose ϵ for the iterative scaling approach to match the accuracy you get from Mosek with its default settings.
5. It is also possible to solve the above maximum entropy problem using a non-conic convex interior-point solver. An especially good solver for this problem can be downloaded at <http://web.stanford.edu/group/SOL/software/pdco/>. Compare the performance of this solver with the two previous ones. Also here try to make the comparison fair with respect to tolerances. Check the link to the presentation at ISMP2003 on the above web page. It seems that they are able to solve maximum entropy problems for web traffic with roughly 50000 nodes using Matlab. Are you able to?